酒店预订平台

项目简介

智慧出行酒店预订平台是一个面向现代旅游出行场景的综合服务体系，旨在为酒店商家与终端消费者之间搭建高效、便捷的信息交互桥梁。

项目结构

```
hotel-booking-platform/
├── client/                 # 用户端（React H5 移动端）
├── admin/                  # 管理端（React PC端）
├── hotel-booking-taro/     # 小程序端（Taro 跨端框架）
├── server/                 # 后端服务
└── README.md
```

技术栈

前端

- **用户端**: React 18 + React Router + Ant Design Mobile
- **管理端**: React 18 + React Router + Ant Design
- **状态管理**: React Hooks + Context API
- **HTTP客户端**: Axios

后端

- **框架**: Node.js + Express
- **数据库**:
  - 默认: 内存数据库
  - **推荐**: MongoDB + Mongoose ODM（✨ 已集成）
  - 可选: MySQL/PostgreSQL
- **认证**: JWT（双 Token）

功能模块

用户端（移动端 / 小程序）

1. **酒店查询页**
   - 顶部Banner广告
   - 地点定位（GPS + IP定位）
   - 关键字搜索
   - 入住日期选择
   - 筛选条件（星级、价格）
   - 快捷标签

    - 支持11个主要城市

2. **酒店列表页**

    - 核心条件筛选
    - 详细筛选
    - 酒店列表（支持分页加载）
    - 多条件组合搜索

3. **酒店详情页**

    - 酒店图片轮播
    - 基础信息展示
    - 日历选择（入住/离店日期）
    - 房型价格列表
    - 🛒 **加入购物车**（一键添加）
    - 实时价格计算

4. **购物车页面** ✨ NEW

    - 查看已选房间
    - 修改数量
    - 删除商品
    - 清空购物车
    - 价格汇总
    - 前往结算

5. **订单管理页面** ✨ NEW

    - 订单列表（全部/待支付/已确认/已支付/已取消）
    - 订单详情查看
    - 支付订单
    - 取消订单
    - 订单状态实时更新

6. **订单结算页面** ✨ NEW

    - 填写入住人信息
    - 联系电话
    - 电子邮箱（可选）
    - 特殊需求备注
    - 一键下单

管理端（PC端）

1. **用户登录/注册**

    - 支持商户和管理员两种角色

2. **酒店信息管理**（商户角色）

    - 酒店信息录入

    - 信息编辑和修改

3. **酒店审核管理**（管理员角色）
    - 信息审核（通过/不通过）
    - 酒店发布
    - 酒店下线

快速开始

📦 MongoDB 设置（推荐）

本项目已集成 MongoDB 支持。如要使用 MongoDB：

```
# 1. 启动 MongoDB (Docker)
docker run -d -p 27017:27017 --name mongodb mongo:latest
# #如果失败，重建容器:
docker rm -f mongodb
docker run -d -p 27017:27017 --name mongodb `
  -e MONGO_INITDB_ROOT_USERNAME=admin `
  -e MONGO_INITDB_ROOT_PASSWORD=admin123 `
  mongo:latest
  然后再测:
  docker run --rm -it mongo:latest mongosh `
  "mongodb://admin:admin123@host.docker.internal:27017/admin?authSource=admin" `
  --eval "db.runCommand({connectionStatus:1})"
直接连容器网络
docker run --rm -it --network container:mongodb mongo:latest mongosh `
  "mongodb://admin:admin123@localhost:27017/admin?authSource=admin" `
  --eval "db.runCommand({connectionStatus:1})"
mongDB CONNECT STRING :mongodb://admin:admin123@localhost:27017/admin
# 2. 配置环境变量
cd server
cp .env.example .env
# 编辑 .env, 设置 MONGODB_URI

# 3. 初始化数据 (可选)
node scripts/seed.js
```

详细说明请参考：

- MongoDB 快速开始指南
- 完整迁移指南
- 完成清单与工作指南

安装依赖

```
# 安装所有依赖
npm run install-all
```

开发模式

```
# 同时启动所有服务
npm run dev

# 或分别启动
npm run dev:client      # 启动用户端（端口3000）
npm run dev:admin       # 启动管理端（端口3001）
npm run dev:server      # 启动后端服务（端口5000）
npm run dev:taro        # 启动小程序开发服务
```

结束占用端口的进程：netstat -ano | findstr :5000 taskkill /PID 29908 /F

Taro 小程序编译

```
cd hotel-booking-taro

# 开发调试
npm run dev:weapp       # 编译微信小程序（开发模式）
npm run dev:alipay      # 编译支付宝小程序（开发模式）
npm run dev:h5          # 编译 H5（开发模式）

# 生产构建
npm run build:weapp     # 编译微信小程序（生产模式）
npm run build:alipay    # 编译支付宝小程序（生产模式）
npm run build:h5        # 编译 H5（生产模式）
```

编译完成后：

- **微信小程序**：用微信开发者工具打开 `hotel-booking-taro/dist/weapp`
- **支付宝小程序**：用支付宝开发者工具打开 `hotel-booking-taro/dist/alipay`
- **H5**：访问本地服务 `http://localhost:10086`

构建生产版本

```
npm run build
```

默认账号

管理员账号

- 用户名: admin
- 密码: admin123

商户账号

- 用户名: merchant1
- 密码: merchant123

API文档

基础路径

- 开发环境: `http://localhost:5000/api`

主要接口

认证

- `POST /api/auth/register` - 用户注册
- `POST /api/auth/login` - 用户登录
- `POST /api/auth/refresh` - 刷新 access token（依赖 httpOnly Cookie）
- `POST /api/auth/logout` - 退出登录（清理 refresh token）
- `GET /api/auth/me` - 获取当前用户信息

酒店信息

- `GET /api/hotels` - 获取酒店列表
- `GET /api/hotels/:id` - 获取酒店详情
- `POST /api/hotels` - 创建酒店（商户）
- `PUT /api/hotels/:id` - 更新酒店（商户）
- `DELETE /api/hotels/:id` - 删除酒店（商户）

审核管理

- `GET /api/admin/hotels/pending` - 获取待审核酒店
- `PUT /api/admin/hotels/:id/approve` - 审核通过
- `PUT /api/admin/hotels/:id/reject` - 审核拒绝
- `PUT /api/admin/hotels/:id/publish` - 发布酒店
- `PUT /api/admin/hotels/:id/unpublish` - 下线酒店

数据字段说明

酒店状态字段

- **status**: 发布状态
  - `draft`: 草稿（商户创建后的初始状态）
  - `published`: 已发布（客户端可见）
  - `unpublished`: 已下线
- **reviewStatus**: 审核状态
  - `pending`: 待审核（商户创建后的初始状态）
  - `approved`: 审核通过（可以发布）
  - `rejected`: 审核拒绝

酒店审核发布流程

1. **商户创建酒店** → status: `draft`, reviewStatus: `pending`
2. **管理员审核通过** → reviewStatus: `approved`（或 `rejected`）
3. **管理员发布酒店** → status: `published`
4. **客户端可见** → 只显示 status 为 `published` 的酒店

查询客户端酒店（只显示published状态）

Invoke-RestMethod http://127.0.0.1:5000/api/hotels | Select-Object -ExpandProperty data | Select-Object nameCn, status, reviewStatus | Format-Table

酒店信息必填字段

- 酒店名称（中英文）
- 酒店地址
- 酒店星级
- 酒店房型 [{"type":"标准间","price":300,"description":"25平米"}]
- 酒店价格
- 开业时间

可选字段

- 附近景点
- 交通信息
- 附近商场
- 优惠信息

评分标准

- 功能完成度：60分
- 技术复杂度：10分
- 用户体验：10分
- 代码质量：10分
- 项目创新性：10分

注意事项

1. 代码需要有清晰的Git提交记录
2. 不要一次性提交所有代码
3. 每个提交应该有明确的意义
4. 可以使用开源UI组件库

双 Token 认证说明

本项目使用 Access Token + Refresh Token 双 Token 机制：

- Access Token：短期有效（默认 15 分钟），通过 Authorization Bearer Header 传递。
- Refresh Token：长期有效（默认 7 天），写入 httpOnly Cookie，仅用于刷新 Access Token。

前端已内置 401 自动刷新逻辑：当 Access Token 过期会调用刷新接口，拿到新 Access Token 后重试原请求。
PS F:\hotel-booking-platform> .\test-double-token.ps1 ========== Double Token Test ========== Server: http://localhost:5000
[Test 1] Login and get tokens... OK Login successful OK Access Token: eyJhbGciOiJIUzI1NiIsInR5cCI6Ik... OK Refresh Token Cookie set in response header - Found Set-Cookie header with refresh_token

[Test 2] Access protected endpoint... OK Access successful - User: merchant - Role: merchant [Test 3] Refresh with Refresh Token... OK Refresh successful OK New Access Token: eyJhbGciOiJIUzI1NiIsInR5cCI6Ik... WARN Token unchanged

[Test 4] Access with new token... OK New token valid

[Test 5] Logout... OK Logout successful OK Refresh token invalidated (expected)

MongoDB 核心特性

✨ 已为项目集成：

- ☑ **Mongoose 数据模型**：User 和 Hotel 模型，包含完整验证和钩子
- ☑ **自动连接管理**：生产级数据库连接配置，支持多环境
- ☑ **密码加密**：自动 bcrypt 哈希处理（pre-save 钩子）
- ☑ **索引优化**：文本索引和复合索引，提升查询性能
- ☑ **数据验证**：Schema 级别的字段验证和类型检查
- ☑ **数据种子脚本**：快速初始化示例数据（4个用户 + 4家酒店）
- ☑ **错误处理**：MongoDB 特定错误处理（ValidationError, E11000 等）
- ☑ **聚合管道**：支持复杂查询（按城市分组、统计等）

快速启动 MongoDB

```
# Docker 启动（推荐）
docker run -d -p 27017:27017 --name mongodb mongo:latest

# 初始化应用
cd server
npm install
cp .env.example .env
# 编辑 .env，设置 MONGODB_URI=mongodb://localhost:27017/hotel-booking-dev

# 初始化数据（可选）
node scripts/seed.js

# 启动服务器
npm run dev
```

💡 **提示**: 详细说明见 MONGODB_QUICK_START.md

开发团队